Master Guide: DSA Problem-Solving Patterns

The definitive, structured structural blueprint organized from Level 1 to Level 4. Optimized with conceptual frameworks, recognition keys, and core linear/non-linear logic mechanics.

🛑 LEVEL 1: BEGINNER FUNDAMENTALS (LINEAR STRUCTURES)

1. Two Pointers

BLUEPRINT	Place two pointer variables at different indexes of an array (usually one at the beginning $left = 0$ and one at the end $right = n - 1$) and move them toward each other based on a condition.
CORE NOTE	Eliminates nested $O(N^2)$ loops by processing two elements at once in a single $O(N)$ linear scan.
WHEN TO USE	When dealing with sorted arrays or strings, and you need to find pairs or compare elements from both ends.

EASY LEETCODE PROBLEMS

> **LeetCode 167 - Two Sum II (Input Array Is Sorted)**
>
> **Hint:** Start pointers at both ends. If the sum is too large, move the right pointer left; if too small, move the left pointer right.

> **LeetCode 125 - Valid Palindrome**
>
> **Hint:** Place one pointer at the start and one at the end, moving inward while comparing characters and skipping non-alphanumeric symbols.

2. Sliding Window

BLUEPRINT	Maintain a continuous sub-segment (the "window") over an array or string. Expand the window by moving a right pointer forward, and shrink it by moving a left pointer forward when a condition is violated.
CORE NOTE	Converts a brute-force $O(N^2)$ subarray search into a smooth $O(N)$ pass by tracking a running state rather than re-summing from scratch.
WHEN TO USE	When the problem asks for a contiguous subarray or substring that matches specific criteria.

EASY LEETCODE PROBLEMS

> **LeetCode 643 - Maximum Average Subarray I**
>
> **Hint:** Create a window of size k, compute its sum, then slide it across the array by adding the next element and subtracting the element left behind.

> **LeetCode 2269 - The K-Beauty of a Number**
>
> **Hint:** Convert the number to a string and use a fixed window of size k to extract substrings, convert them back to integers, and check if they divide the original number.

3. Fast & Slow Pointers (Hare & Tortoise)

BLUEPRINT	Initialize two pointers at the head of a sequence. The slow pointer moves forward by 1 step ($slow = slow.next$), while the fast pointer moves forward by 2 steps ($fast = fast.next.next$).
CORE NOTE	If a cycle exists, the fast pointer will eventually lap and meet the slow pointer. If no cycle exists, the fast pointer hits the end of the data.
WHEN TO USE	Specifically for cyclic data structures (Linked Lists or pointer-based arrays) to find loops or structural midpoints.

EASY LEETCODE PROBLEMS

> **LeetCode 141 - Linked List Cycle**
>
> **Hint:** Move fast by two and slow by one. If they point to the exact same node at any point, a cycle exists.

> **LeetCode 876 - Middle of the Linked List**
>
> **Hint:** By the time the fast pointer reaches the end of the list, the slow pointer will be standing exactly at the middle node.

4. Bitwise XOR

BLUEPRINT	Utilizing the logical properties of the XOR (⊕) operator where $A \oplus A = 0$ (elements cancel out) and $A \oplus 0 = A$ (element remains).
CORE NOTE	Allows you to find missing or unique values in $O(N)$ time and $O(1)$ extra space without using HashMaps.
WHEN TO USE	When an array contains elements where almost all appear an even number of times, and you need to find the lone exception.

EASY LEETCODE PROBLEMS

> **LeetCode 136 - Single Number**
>
> **Hint:** XOR all numbers in the array together. The duplicates cancel each other out to 0, leaving behind the unique number.

> **LeetCode 268 - Missing Number**
>
> **Hint:** XOR all numbers from 0 to n, and then XOR that result with all numbers present in the array. The remaining value is your missing number.

🔶 LEVEL 2: INTERMEDIATE SORTING & DATA MANAGEMENT

5. Modified Binary Search

BLUEPRINT	An extension of traditional binary search ($low + (high - low) / 2$). It adapts to conditions where the array is altered but still retains a logical split property.
CORE NOTE	Always yields an incredibly fast $O(\log N)$ time complexity.
WHEN TO USE	When searching for an element in a dataset that is sorted, or contains distinct "peak/valley" boundary properties.

EASY LEETCODE PROBLEMS

> **LeetCode 704 - Binary Search**
>
> **Hint:** The textbook pattern. Compare the middle element to the target and discard the left or right half of the sorted array accordingly.

> **LeetCode 35 - Search Insert Position**
>
> **Hint:** Use standard binary search. If the target isn't found, the low pointer index at the end of the loop is exactly where the target should be inserted.

---

6. Merge Intervals

BLUEPRINT	Sort an array of intervals by their starting values, then iterate through them sequentially checking if the start time of the current interval overlaps with the end time of the previous one.
CORE NOTE	Vital for resolving chronological overlaps or gaps in structural scheduling.
WHEN TO USE	Any problem involving time slots, calendar schedules, or overlapping numerical ranges.

EASY LEETCODE PROBLEMS

> **LeetCode 252 - Meeting Rooms**
> **Hint:** Sort the meetings by start time. Loop through and check if any meeting starts before the previous one ends.

> **LeetCode 228 - Summary Ranges**
> **Hint:** Iterate through a sorted array and identify continuous intervals of numbers where ***nums[i] + 1 == nums[i+1]***, grouping them into ranges.

---

7. In-place Reversal of a Linked List

BLUEPRINT	Alter the directional links (***node.next***) of a Linked List directly using three tracking pointers: ***prev***, ***current***, and ***next***.
CORE NOTE	Reverses nodes without allocating any extra memory nodes, keeping space complexity at a strict ***O(1)***.
WHEN TO USE	When asked to reverse an entire linked list or a specific segment of it.

EASY LEETCODE PROBLEMS

> **LeetCode 206 - Reverse Linked List**
> **Hint:** Loop through the list, temporarily save ***current.next***, point ***current.next*** backwards to ***prev***, then move ***prev*** and ***current*** forward.

> **LeetCode 234 - Palindrome Linked List**
> **Hint:** Use fast/slow pointers to find the middle, reverse the second half of the list in-place, and compare it to the first half.

---

8. Top 'K' Elements (Heap Pattern)

BLUEPRINT	Use a Heap (Priority Queue) to track elements dynamically. To find the largest elements, insert numbers into a Min-Heap and keep its size capped at K, popping out the smallest values.
CORE NOTE	Cuts the time complexity down from a full sort ($O(N \log N)$) to an efficient $O(N \log K)$.
WHEN TO USE	Whenever a problem asks for the "K largest," "K smallest," or "K most frequent" items.

EASY LEETCODE PROBLEMS

> **LeetCode 703 - Kth Largest Element in a Stream**
>
> **Hint:** Maintain a Min-Heap of size K. The top element of the heap will always represent the K-th largest element currently in the stream.

> **LeetCode 1046 - Last Stone Weight**
>
> **Hint:** Put all stone weights into a Max-Heap. Repeatedly poll the top two heaviest stones, smash them, and put the remaining weight back into the heap until one or zero stones remain.

---

9. Two Heaps

BLUEPRINT	Maintain a Max-Heap to store the smaller half of incoming numbers and a Min-Heap to store the larger half. Keep the sizes balanced (difference in size ≤ 1).
WHEN TO USE	Situations where elements arrive continuously as a dynamic stream, and you need to continuously locate structural balance points (like the median).

EASY LEETCODE PROBLEMS (FOUNDATIONAL MECHANICS)

> **LeetCode 506 - Relative Ranks**
>
> **Hint:** Use a Max-Heap to extract players with the highest scores one by one to assign them "Gold Medal", "Silver Medal", etc., based on placement.

> **LeetCode 1464 - Maximum Product of Two Elements in an Array**
>
> **Hint:** Use a Max-Heap to instantly grab the two largest unique integers from an array to calculate their product.

---

10. K-way Merge

BLUEPRINT	When given K sorted collections, push the first element of each into a Min-Heap. Pop the smallest element to place into your final sorted list, and then insert the next element from that same origin array.
WHEN TO USE	Merging multiple pre-sorted arrays, lists, or data streams into one single sorted output.

EASY LEETCODE PROBLEMS (BUILDING BLOCKS)

> **LeetCode 88 - Merge Sorted Array**
> **Hint:** A 2-way merge. Start from the back of both arrays and place the larger elements at the end of the target array to avoid overwriting data.

> **LeetCode 21 - Merge Two Sorted Lists**
> **Hint:** Create a dummy head node, use pointers to compare the heads of two sorted linked lists, and link the smaller node next sequentially.

🌲 LEVEL 3: NON-LINEAR & HIERARCHICAL TRAVERSAL

11. Breadth-First Search (BFS)

BLUEPRINT	Traverse a tree or graph level-by-level. Use a Queue data structure to keep track of nodes. Visit a node, pop it, and push all its immediate unvisited children into the queue before moving deeper.
CORE NOTE	Guarantees finding the absolute shortest path first in an unweighted graph structure.
WHEN TO USE	Level-order tree traversals, or finding minimum steps to a target in a graph.

EASY LEETCODE PROBLEMS

> **LeetCode 637 - Average of Levels in Binary Tree**
> **Hint:** Use a queue to perform BFS. For each level, track the number of nodes, sum up their values, calculate the average, and push it to your result.

> **LeetCode 104 - Maximum Depth of Binary Tree (BFS Approach)**
> **Hint:** Run a level-order traversal queue loop. Increment your depth counter by 1 every time you completely finish emptying a full layer from the queue.

12. Depth-First Search (DFS)

BLUEPRINT	Traverse down a single structural path as deep as possible until you hit a leaf node, then backtrack to explore the next branch. Implemented cleanly using Recursion.
WHEN TO USE	Path-finding, checking structural properties, or calculating tree diameters.

EASY LEETCODE PROBLEMS

> **LeetCode 112 - Path Sum**
> **Hint:** Recursively traverse down the tree, subtracting the current node's value from targetSum. Check if a leaf node's remaining value equals zero.

> **LeetCode 226 - Invert Binary Tree**
> **Hint:** At each node, recursively swap its left child and right child, then call the function on both children.

---

13. Island Pattern (Matrix DFS/BFS)

BLUEPRINT	Treat a 2D grid/matrix as a graph where each cell (r, c) is a node, and its 4 adjacent directions (up, down, left, right) are connected edges.
WHEN TO USE	When asked to find connected components, clusters, or boundaries inside a 2D grid array.

EASY LEETCODE PROBLEMS

> **LeetCode 733 - Flood Fill**
> **Hint:** Use DFS/BFS starting at the target cell. Change its color, then recursively invoke the change on all adjacent matching-color pixels.

> **LeetCode 463 - Island Perimeter**
> **Hint:** Loop through the grid. When you find a land cell, check its 4 neighbors. If a neighbor is water or out-of-bounds, add 1 to your perimeter count.

---

14. Topological Sort

BLUEPRINT	Linear ordering of vertices in a Directed Acyclic Graph (DAG) such that if an edge points from $u \rightarrow v$, u comes before v. Implemented by tracking incoming dependencies ("in-degrees").
WHEN TO USE	Managing tasks with strict prerequisite chains or order-of-operation assignments.

EASY LEETCODE PROBLEMS (GRAPH DEPENDENCY FUNDAMENTALS)

> **LeetCode 1791 - Find Center of Star Graph**
> **Hint:** The center node must appear in every single edge. Compare the first two edges; the vertex common to both is your center.

> **LeetCode 1971 - Find if Path Exists in Graph**
> **Hint:** Build an adjacency list from the edge pairs, then use a basic DFS/BFS traversal or a Disjoint Set Union (DSU) to check if the source connects to the destination.

---

15. Trie (Prefix Tree)

BLUEPRINT	A specialized tree structure where each node represents a character of a string. Walking down a path from the root spells out full words.
WHEN TO USE	Storing strings, performing prefix matching, auto-complete, or dictionary lookups.

EASY LEETCODE PROBLEMS (STRING/PREFIX MATCHING BASICS)

> **LeetCode 14 - Longest Common Prefix**
> **Hint:** Sort the array of strings and compare characters of the first and last string from left to right until they mismatch.

> **LeetCode 2185 - Counting Words With a Given Prefix**
> **Hint:** Loop through each word in the array and use string matching operations like *.startsWith()* to check if the prefix matches.

🔥 LEVEL 4: ADVANCED ALGORITHMIC FRAMEWORKS

16. Subsets & Combinations (Backtracking)

BLUEPRINT	A state-space search pattern. You build a solution step-by-step using recursion. If a choice leads to a dead-end or violates constraints, you "backtrack" by undoing your last choice.
WHEN TO USE	Generating permutations, combinations, or searching all valid decision trees.

EASY LEETCODE PROBLEMS (ENTRY-LEVEL TREE PATHS)

> **LeetCode 257 - Binary Tree Paths**
>
> **Hint:** Traverse the tree using recursion. Add the current node to your path string. When you hit a leaf, append it to your result list. Backtrack out of the recursive call to explore other paths.

> **LeetCode 1863 - Sum of All Subset XOR Totals**
>
> **Hint:** For each number in the array, you have two choices: include it in the current subset or exclude it. Recursively generate all subsets and sum up their XOR totals.

---

17. Dynamic Programming (DP)

BLUEPRINT	Solve complex optimization problems by breaking them down into overlapping subproblems. Save each subproblem's answer in a lookup table (Memoization or Tabulation) to avoid re-solving it.
WHEN TO USE	Problems asking for maximum profit, minimum cost, or total unique ways to reach a target based on previous states.

EASY LEETCODE PROBLEMS

> **LeetCode 70 - Climbing Stairs**
>
> **Hint:** To reach step n, you could only have come from step $n-1$ or step $n-2$. Thus, $ways[n] = ways[n-1] + ways[n-2]$ (exactly like the Fibonacci sequence).

> **LeetCode 746 - Min Cost Climbing Stairs**
>
> **Hint:** Maintain an array where the minimum cost to reach step i is $cost[i] + min(dp[i-1], dp[i-2])$. Build this iteratively from the bottom up.